

Flutter App Development Course Syllabus

Archer Infotech, Kothrud Pune | Classroom & live-online batches | Updated September 2026

Institute: Archer Infotech, Kothrud, Pune - IT training since 2009

Website: <https://archerinfotech.in>

Contact: +91 9850 678451 | info@archerinfotech.in

Mode: Classroom at Kothrud, Pune and live-online batches

Duration: 3.5 Months

Level: Intermediate

Course page: <https://archerinfotech.in/courses/mobile-app-development/flutter-development-training-in-pune>

Course Overview

Flutter is the dominant cross-platform mobile framework in Pune startup hiring — single codebase ships to iOS, Android, web, and increasingly desktop, with native-class performance via the Dart-to-AOT compilation. Pune fintech and consumer-tech companies (BharatPe Pune for some apps, Razorpay Pune for some clients, Drip Capital, plus most Pune fintech / health-tech / ed-tech early-stage startups) ship in Flutter to maximise reach with limited mobile-team headcount. Archer Infotech's Flutter Development training in Pune teaches the framework as it is actually used in 2026 — Flutter 3.x with Dart 3.x null-sound safety, Material 3 widgets, Cupertino widgets for iOS-feel, state management with Riverpod (the modern default) plus Bloc for the production / large-app case, plus Firebase integration and the production tail (signing, App Store / Play Store deployment). Classroom in Kothrud, online live, and weekend batches available.

Why Learn Flutter in 2026

Flutter has eclipsed React Native as the dominant cross-platform framework in Indian startup hiring — Indeed Pune lists more than 400 active Flutter Developer openings as of May 2026, with continuous hiring at BharatPe Pune, Drip Capital, Pune-based ed-tech / health-tech startups, plus several BFSI mobile teams that have adopted Flutter for new apps. The single-codebase economics are compelling — for early-stage and mid-stage companies, one Flutter team replaces two native (Android + iOS) teams.

What changed in 2026: Dart 3.x has matured (sound null safety, pattern matching, records, sealed classes, plus the new macro system). Flutter 3.x has stabilised across iOS / Android / Web / desktop. The Impeller rendering engine (replacing Skia for iOS) has shipped with strong performance. Material 3 widgets are the default; Cupertino widgets for the iOS-platform look. State management has settled — Riverpod (the modern default for new projects) for most cases, Bloc for the production / large-app case, GetX for legacy code. Firebase remains the de-facto backend for Flutter apps.

What this means for hiring: 2026 Pune Flutter JDs expect Dart 3.x fluency, Flutter 3.x at depth, Riverpod or Bloc, Firebase integration, plus one published app on either Play Store or App Store. Senior roles add Custom Painter / RenderObject for advanced UI, platform channels for native integration, and CI/CD with Codemagic or Bitrise.

- 400+ active Flutter Developer roles on Indeed Pune (May 2026)
- Dominant cross-platform framework in Indian startup hiring
- Single codebase ships to iOS + Android + Web + Desktop
- Riverpod + Bloc — the modern state-management defaults
- Strong Pune fintech / consumer-tech / startup hiring

Who This Course Is For

This course fits you if

- Engineering, BCS, MCA, or BSc-CS student targeting cross-platform mobile roles

- Working web developer (any stack) wanting to switch to mobile with maximum cross-platform reach
- Working Android-only or iOS-only developer wanting to add the other platform via Flutter
- Working Java / Kotlin / Swift developer wanting to add Dart for startup hiring
- Founder / solopreneur shipping a mobile MVP — Flutter is the right framework for solo / small teams
- Career restarter targeting the Pune startup mobile-development scene

This course is not the right fit if

- If you have no programming background — at least basic Java / Kotlin / JavaScript / Python is required
- If your goal is platform-specific deep features (iOS Apple Watch, Android Wear, ARKit / ARCore at depth) — pick native iOS or Android
- If you cannot put in 8–10 hours per week of practice outside class
- If you only want a certificate sticker — Pune Flutter hiring screens hard on real apps and GitHub
- If you have 3+ years of production Flutter experience — talk to us about advanced specialisations

Detailed Curriculum

Module 1: Dart 3 Language Foundations (Week 1)

Dart taught as a language before any widget appears. Variables and types, sound null safety and what late really promises, control flow, functions with named and optional parameters, classes, mixins and extension methods.

Dart 3 gets proper coverage because it changed how idiomatic Dart reads: records for lightweight multiple returns, pattern matching and destructuring, sealed classes with exhaustive switch expressions, and class modifiers. Asynchrony closes the week — Future, async/await, Stream, and the difference between a single-subscription and a broadcast stream.

- Variables, types and sound null safety
- late, required and nullable-aware operators
- Functions, named and optional parameters
- Classes, mixins and extension methods
- Records for lightweight multiple returns
- Pattern matching and destructuring
- Sealed classes and exhaustive switch expressions
- Future, async / await and error handling
- Stream types — single-subscription vs broadcast

Module 2: Flutter Tooling & Project Anatomy (Week 2)

The toolchain and the shape of a Flutter project. SDK installation with flutter doctor resolved properly on Windows and macOS, IDE setup in Android Studio or VS Code, and emulator plus physical-device runs on both platforms.

Then the project itself: pubspec.yaml, dependency and dev-dependency management, version constraints, assets and fonts, and platform folders. Hot reload versus hot restart is taught with its actual limits — what state survives, what does not, and why a const constructor sometimes stops a reload appearing.

- Flutter SDK install and resolving flutter doctor
- Android Studio and VS Code setup
- Emulator, simulator and physical-device runs
- pubspec.yaml, dependencies and version constraints
- Assets, fonts and platform folders
- Hot reload vs hot restart and their limits
- DevTools widget inspector
- Flavours for dev, staging and production
- Linting with flutter_lints and analysis_options

Module 3: Widgets, Layout & the Widget Tree (Week 3)

Flutter is widgets all the way down, and the tree is the mental model everything else depends on. StatelessWidget versus StatefulWidget, the build method, BuildContext and what it can and cannot reach, the element tree, and keys —

including the classic bug where reordering a list scrambles state because keys were omitted.

Layout follows: Column, Row, Stack, Container, Padding, Expanded and Flexible, Wrap, and the constraint model — constraints go down, sizes go up, parent sets position — which is the sentence that resolves most layout confusion.

- StatelessWidget vs StatefulWidget
- build, BuildContext and the element tree
- Keys and the list-reordering state bug
- Column, Row, Stack and Container
- Expanded, Flexible and Spacer
- Constraints go down, sizes go up
- Debugging overflow and unbounded-height errors
- ListView, GridView and builder constructors
- SingleChildScrollView and scroll physics

Module 4: Material 3, Cupertino, Theming & Responsive Layout (Week 4)

Making an app look right on both platforms and on every screen size. The Material 3 widget set as the default, Cupertino widgets where an iOS-native feel is wanted, and an honest position on adaptive versus uniform design.

Theming covers ThemeData, colour schemes, typography, component themes and dynamic colour, plus light and dark mode. Responsive layout uses LayoutBuilder, MediaQuery, OrientationBuilder and breakpoints for phone, tablet and foldable, with safe areas, text scaling and accessibility handled as requirements.

- Material 3 widget set in practice
- Cupertino widgets and platform-adaptive design
- ThemeData, colour schemes and typography
- Component themes and design tokens
- Light, dark and dynamic colour
- LayoutBuilder, MediaQuery and OrientationBuilder
- Breakpoints for phone, tablet and foldable
- SafeArea, notches and system insets
- Text scaling, semantics and accessibility

Module 5: State Management with Riverpod (Week 5)

State management as Pune Flutter teams actually do it, starting with why setState stops scaling. Riverpod is the modern default for new projects: providers and their variants, ConsumerWidget and ref, AsyncValue for representing loading, data and error in one type, and the AsyncNotifier pattern.

Code generation with riverpod_generator removes the boilerplate and gives compile-time safety. The module also covers provider scoping and overrides — the mechanism that makes Riverpod genuinely testable — plus dependency injection and caching with keepAlive.

- Why setState stops scaling
- Providers and provider variants
- ConsumerWidget, ConsumerStatefulWidget and ref
- AsyncValue — loading, data and error in one type
- Notifier and AsyncNotifier patterns
- riverpod_generator and compile-time safety
- Scoping, overrides and testability
- Caching, keepAlive and invalidation
- Error handling and retry patterns

Module 6: Bloc & Cubit for Large Applications (Week 6)

The pattern a large share of Pune Flutter teams standardise on, and the one most likely to appear in an interview. Cubit first as the simpler form — states as immutable classes, emit, and BlocBuilder. Then full Bloc with events, event transformers for debounce and throttle, and BlocListener versus BlocConsumer for side effects.

State modelling gets real attention: sealed state classes with exhaustive handling, `Equatable` for correct rebuild behaviour, and `BlocObserver` for logging. The module ends with an honest comparison of Riverpod and Bloc, and guidance on choosing per team.

- Cubit — states, emit and `BlocBuilder`
- Bloc — events, handlers and transformers
- Event transformers for debounce and throttle
- `BlocListener`, `BlocConsumer` and side effects
- Sealed state classes and exhaustive handling
- `Equatable` and correct rebuild behaviour
- `BlocProvider`, `MultiBlocProvider` and scoping
- `BlocObserver` for logging and debugging
- Riverpod vs Bloc — choosing per team

Module 7: Navigation with `go_router` & Deep Linking (Week 7)

Routing that survives an app growing past ten screens. `go_router` as the modern default replacing hand-rolled `Navigator` calls: declarative route definitions, path and query parameters, nested and shell routes for persistent bottom navigation, and typed routes via code generation.

Redirection and route guards handle authentication flows, including the redirect-after-login pattern every real app needs. Deep linking closes the module — custom schemes, Android App Links and iOS Universal Links, with the platform configuration and verification each requires.

- `go_router` route definitions and the router delegate
- Path parameters, query parameters and extra
- Nested and shell routes for persistent navigation
- Typed routes with code generation
- Redirection and authentication guards
- The redirect-after-login pattern
- Custom URL schemes
- Android App Links and iOS Universal Links
- Handling a cold start from a deep link

Module 8: Forms, Validation & User Input (Week 8)

Input handling done to a production standard. The `Form` widget and `GlobalKey<FormState>`, `TextFormField`, controllers and focus nodes, and validation that runs at the right moment rather than on every keystroke.

Complex forms use `flutter_form_builder` or a reactive-forms approach, with cross-field validation and async validation against a backend. The module covers keyboard handling and the overflow it causes, input formatters and masks for phone and PAN entry, and accessibility — labels, focus order and error announcement.

- `Form`, `GlobalKey<FormState>` and `TextFormField`
- Controllers, focus nodes and focus traversal
- Validation timing and `autovalidate` modes
- `flutter_form_builder` for complex forms
- Cross-field and async validation
- Keyboard handling and viewport insets
- Input formatters and masks
- Error display and accessibility announcement
- Multi-step and wizard forms

Module 9: Networking — Dio, REST & GraphQL (Week 9)

The data layer's outward half. `http` for simple cases and `Dio` where interceptors, cancellation and upload progress matter — authentication and retry interceptors, timeouts tuned for poor networks, and structured error mapping.

JSON serialisation uses `json_serializable` with `build_runner`, including nested models and schema-mismatch tolerance. GraphQL is covered with `graphql_flutter` for teams on that stack, and the module ends on file upload and download, and mocking HTTP for tests.

- http for simple requests, Dio for real apps
- Interceptors for auth, logging and retry
- Timeouts and tuning for poor networks
- Cancellation tokens and request lifecycle
- `json_serializable` and `build_runner`
- Nested models and schema-mismatch tolerance
- Mapping failures to typed error objects
- GraphQL with `graphql_flutter`
- File upload, download and progress reporting

Module 10: Local Persistence & Offline-First (Week 10)

Data that outlives the process. Drift as the typed Dart ORM over SQLite — tables, queries, joins, migrations and reactive streams that push updates into the UI. Isar as the NoSQL alternative, `shared_preferences` for simple key-value, and `flutter_secure_storage` for tokens.

Offline-first is the point of the module: the local database as single source of truth, a repository that reconciles remote and local, queued mutations while offline, conflict resolution, and connectivity awareness — the design Indian consumer apps need on unreliable networks.

- Drift tables, queries, joins and migrations
- Reactive streams from the database
- Isar as the NoSQL alternative
- `shared_preferences` and `flutter_secure_storage`
- Local database as single source of truth
- Repository reconciling remote and local
- Queued mutations and replay on reconnect
- Conflict resolution strategy
- `connectivity_plus` and network awareness

Module 11: Firebase & Realtime Backends (Week 10)

Firebase as the backend most Flutter projects reach for first. FlutterFire setup across both platforms, Firebase Auth with email, Google and phone-number sign-in — the last dominant in the Indian market — and Firestore data modelling with real-time listeners.

Security rules are written and tested rather than left permissive. Cloud Storage handles user media, Cloud Messaging handles push including foreground, background and terminated-state delivery, and Crashlytics, Analytics and Remote Config close the module with a real crash report read and fixed.

- FlutterFire setup on Android and iOS
- Firebase Auth — email, Google, phone number
- Firestore modelling and real-time listeners
- Security rules and testing them
- Cloud Storage for user media
- Cloud Messaging — foreground, background, terminated
- Crashlytics and reading a real crash report
- Analytics events and Remote Config flags
- Cloud Functions triggers overview

Module 12: Animations, Motion & Custom Painting (Week 11)

The layer that makes an app feel considered rather than merely functional. Implicit animations (`AnimatedContainer`, `AnimatedOpacity`, `TweenAnimationBuilder`) for the common cases, then explicit animation with `AnimationController`, `Tween` and curves, and staggered sequences.

Hero transitions connect screens, and CustomPainter with Canvas covers charts, progress indicators and shapes no widget provides. Rive and Lottie integrate designer-produced motion, and the module ends on performance — keeping animation off the build phase and respecting reduced-motion settings.

- Implicit animations and TweenAnimationBuilder
- AnimationController, Tween and curves
- Staggered and sequenced animations
- Hero transitions between screens
- Custom page-route transitions
- CustomPainter and Canvas drawing
- Rive and Lottie integration
- Keeping animation off the build phase
- Respecting reduced-motion accessibility settings

Module 13: Platform Channels, FFI & Native Integration (Week 12)

Reaching past Dart when the platform requires it. Method channels for request-response calls into Kotlin and Swift, event channels for native-to-Dart streams, and the serialisation and threading rules that make channel code correct.

Pigeon generates type-safe channel bindings and removes the string-keyed fragility of hand-written ones. Dart FFI calls C libraries directly, and the module covers plugin package structure, federated plugins, platform permissions on each side, and the practical question of when to write a plugin versus find one.

- Method channels into Kotlin and Swift
- Event channels for native-to-Dart streams
- Serialisation and threading rules
- Pigeon for type-safe channel bindings
- Dart FFI for C libraries
- Plugin package structure and federated plugins
- Platform permissions on Android and iOS
- Camera, location and sensor access
- When to write a plugin versus adopt one

Module 14: Testing, DevTools & Performance (Week 13)

Verifying and tuning what you built. Unit tests with the test package, widget tests with flutter_test including pumping, finders and matchers, golden tests for visual regressions, and integration tests via integration_test on a real device.

Mocking with mocktail and fakes for repositories keeps tests fast. Performance covers DevTools timeline and CPU profiling, the Impeller renderer, diagnosing jank and shader compilation stalls, const constructors and rebuild scope, list virtualisation and image caching, and memory-leak hunting.

- Unit tests with the test package
- Widget tests — pump, finders and matchers
- Golden tests for visual regression
- Integration tests on a real device
- mocktail and repository fakes
- DevTools timeline and CPU profiling
- Impeller and shader compilation jank
- const constructors and rebuild scope
- List virtualisation and image caching
- Finding and fixing memory leaks

Module 15: Release Engineering, CI/CD & Store Publishing (Week 13)

Shipping to two stores from one codebase. Android release builds — keystore signing, App Bundle output, R8 and Dart obfuscation, and the Play Console flow with its Data Safety form and target-API requirements. iOS release — certificates, provisioning profiles, App Store Connect, TestFlight and the review-rejection causes that catch first-time submitters.

Build flavours and environment configuration keep dev, staging and production separate. CI/CD with Codemagic, Bitrise or GitHub Actions automates build, test, sign and upload for both platforms.

- Android keystore signing and App Bundle output
- R8 and Dart obfuscation with symbol files
- Play Console listing, Data Safety and target API
- iOS certificates and provisioning profiles
- App Store Connect, TestFlight and review causes
- Build flavours and environment configuration
- Codemagic, Bitrise and GitHub Actions pipelines
- Automated versioning and build numbers
- Staged rollouts and phased release

Module 16: Capstone Project & Placement Preparation (Weeks 14–15 + 1 week placement prep)

Full-time capstone work followed by structured interview preparation. You build and release a complete cross-platform application — Riverpod or Bloc state management, go_router navigation, a Dio data layer with Drift-backed offline-first storage, Firebase auth and push, animations, tests, and signed builds on both stores' test tracks.

Interview preparation matches Pune Flutter panels: a Dart and async round, a widget-tree and rendering round, a state-management round where you defend your choice, and a system-design round on offline sync. Resume, LinkedIn and GitHub polish included.

- Capstone implementation, release and README
- Publishing to internal test tracks on both stores
- Dart and async interview round
- Widget tree, keys and rendering round
- State-management round — defending your choice
- Mobile system design — offline, sync, pagination
- Resume and LinkedIn rewrite for Flutter JDs
- GitHub portfolio polish
- HR mock interview and salary negotiation

Module 17: AI-Assisted Development Workflow (Final Week)

The skill every 2026 hiring panel now probes for — building real work with AI in the loop, responsibly. Learn to drive AI assistants (GitHub Copilot, Claude, Cursor, and IDE-native AI) to scaffold and accelerate the tools and stack this course covers, generate tests, explain and refactor unfamiliar code, and cut the boilerplate — while keeping you firmly in control of every decision. Heavy focus on guardrails: reviewing each AI suggestion, spotting hallucinated APIs or wrong answers, and handling licensing and data-privacy concerns. Close with a mini-project that takes a deliverable end-to-end using an AI-assisted workflow, then fold the same tooling into version control and everyday team practice.

- AI assistants — GitHub Copilot, Claude, Cursor, IDE-native AI
- Effective prompting for this course's stack — scaffolding, boilerplate, config
- AI-assisted test generation and coverage
- Explaining, refactoring, and modernising unfamiliar code with AI
- AI-driven review, error detection, and quality checks
- Generating and maintaining documentation with AI
- AI debugging — interpreting errors, logs, and failing output
- Guardrails — reviewing output, avoiding hallucinations, licensing & data privacy
- Team workflow — AI in the editor, in reviews, and in delivery pipelines
- Mini-project — a deliverable built end-to-end with an AI-assisted workflow

Capstone Projects

Production Cross-Platform App with Riverpod + Firebase

A complete production-grade Flutter app shipped to both iOS and Android — pick a real domain (expense tracker, recipe finder, fitness tracker, news reader, BFSI-style finance dashboard). Material 3 + Cupertino widgets for platform-appropriate UI, Riverpod for state, Drift for offline storage, Firebase Auth + Firestore + FCM, plus widget + integration tests. Published to Play Store internal-test track + TestFlight (Apple developer fee paid by student — ~Rs. 8,500 / year if iOS publishing is desired; Android Play fee ~Rs. 2,500 one-time). Outcome: a public GitHub repository plus deployment URLs.

- Technology: Flutter 3.x + Dart 3.x
- Technology: Material 3 + Cupertino widgets
- Technology: Riverpod state management
- Technology: Drift ORM + Firebase
- Technology: Widget + integration tests
- Technology: Codemagic CI/CD

Real-Time / Streaming-Heavy App

A real-time streaming app — chat, collaborative whiteboard, or live-feed app. WebSocket or Firestore for real-time, optimistic UI, offline message queue, push notifications. Demonstrates the patterns Pune fintech / consumer-app teams hire on for real-time / collaboration features.

- Technology: Flutter + Riverpod / Bloc
- Technology: WebSocket or Firestore real-time
- Technology: FCM push notifications
- Technology: Offline-first patterns

Cross-Platform App with Native Platform Channels

A Flutter app that meaningfully integrates with native iOS / Android — biometric auth (BiometricPrompt on Android, LocalAuthentication on iOS), platform-specific notifications, sensor data (accelerometer, location), plus a native-only feature exposed via platform channels. Demonstrates the patterns senior Flutter interviews test for.

- Technology: Flutter + platform channels
- Technology: Biometric auth on both platforms
- Technology: Platform-specific notifications
- Technology: Sensor APIs

Career Outcomes

Flutter Developer is the dominant cross-platform mobile role in Pune startup hiring — Indeed Pune lists 400+ active openings, with continuous hiring at BharatPe Pune, Drip Capital, plus the broader Pune fintech / health-tech / ed-tech startup scene. Compensation tracks Native Android within $\pm 5\%$ at every band; senior Flutter engineers at fintech often earn a small premium because the productivity multiplier (one team for both platforms) is meaningful.

What pulls a Flutter developer above the median band: depth on Riverpod or Bloc, demonstrable platform-channel experience, one published app to both stores, plus testing discipline (widget + integration tests). Our capstone projects are designed exactly around these signals.

Senior Flutter Developer bands at the top end are reported as national figures (Pune-specific Indeed pages do not exist for these specific titles); Pune trends within $\pm 10\%$ of these figures.

Roles after this course

- Flutter Developer
- Cross-Platform Mobile Developer
- Mobile App Developer
- Junior Mobile Engineer
- Mobile + Backend Engineer (with Dart server-side)

Indicative salary bands

- Flutter Developer (Pune) - Rs. 6,80,000 per year average (source: Indeed Pune (Flutter Developer))

- Junior Flutter Developer (Pune entry, <2 years) - Rs. 3,50,000 – Rs. 6,50,000 per year (source: AmbitionBox Pune Flutter Developer)
- Mid-level Flutter Developer (Pune, 3–5 years) - Rs. 10,00,000 – Rs. 17,00,000 per year (source: Glassdoor Pune Flutter Developer)
- Senior Flutter Developer (Pune, 5–8 years) - Rs. 16,00,000 – Rs. 28,00,000 per year (source: Glassdoor Pune Senior Flutter Developer)
- Lead Flutter Engineer (national, 8+ years) - Rs. 26,00,000 – Rs. 45,00,000 per year (source: 6figr India Lead Flutter Engineer (Pune ±10%))

Companies hiring for these skills

- BharatPe (Pune)
- Drip Capital
- Innovaccer (Pune)
- Whatfix (Pune)
- Razorpay (Pune)
- Pine Labs (Pune)
- Bajaj Finserv (mobile teams)
- Pune-based ed-tech / health-tech / fintech startups
- TCS
- Infosys
- Cognizant
- Capgemini
- Atos / Eviden

Batch Modes and Duration

Duration: 12 weeks of structured curriculum plus 1 week of capstone and interview preparation (~3.5 months total)

Classroom

- Location: Archer Infotech, Kothrud, Pune
- Morning batch — 10:00 to 13:00
- Evening batch — 18:00 to 21:00

Live online

- Same hours as classroom batches
- Recordings available for review
- Zoom for live sessions
- GitHub for code reviews
- Flutter SDK + Android Studio / VS Code
- Slack / WhatsApp for async Q&A

Weekend

- Saturday + Sunday, 09:00 to 13:00
- Stretches over ~6 months instead of 3.5.

Maximum 15 students per batch.

Fees

Course fees range Rs. 20,000 – Rs. 90,000 depending on mode and concession. Play Console fee (~Rs. 2,500 one-time) and optional Apple Developer Program fee (~Rs. 8,500 / year, only if iOS publishing) are paid by the student.

Indicative range: Rs. 20,000 – Rs. 90,000

- Single payment with early-bird discount
- EMI in 2–3 instalments at no extra cost

- Corporate sponsorship — invoiced with GST

Placement Support

Placement support starts from week 9. By the end of the curriculum your resume highlights real published Flutter apps on both stores, your GitHub has at least two production-style repositories, and you have completed at least three mock technical interviews.

We say placement support, not placement guarantee. Our support is unconditional, time-bound (six months after course completion), and includes free re-entry to a future batch's interview-prep sessions.

The placement process

- Week 9 — resume + LinkedIn rewrite for Flutter JDs
- Week 10 — GitHub portfolio cleanup, store listings
- Weeks 11–12 — three rounds of mock technical interviews
- Week 13 — HR mock and salary negotiation coaching
- Post-course — referrals via our 17-year alumni network at 12+ partner companies (with extra emphasis on Pune startup scene)
- Up to 6 months of continued support
- Free re-entry to future batch interview-prep sessions

Partner companies

- BharatPe (Pune)
- Drip Capital
- Innovaccer (Pune)
- Whatfix (Pune)
- Razorpay (Pune)
- Pine Labs (Pune)
- TCS
- Infosys
- Cognizant
- Capgemini

Prerequisites and How to Start

Prerequisites: at least basic programming background (Java / Kotlin / JavaScript / Python — any), basic OOP understanding, willingness to commit 8–10 hours per week of practice outside class.

Suggested first steps

- Decide your mode — classroom, online live, or weekend
- Check the upcoming batch dates
- Book a free 30-minute counselling call
- Confirm enrolment and complete pre-course orientation (Flutter SDK install, Android Studio / VS Code setup)
- Show up to day one with a laptop running 64-bit OS, 16GB+ RAM (recommended), 50GB+ free disk; macOS preferred if iOS publishing is planned

Frequently Asked Questions

How long does Flutter training in Pune take at Archer Infotech?

Approximately 3.5 months — 12 weeks of structured curriculum plus 1 week of capstone. The weekend batch stretches over ~6 months at the same content depth.

What is the salary of a Flutter Developer in Pune?

Indeed Pune Rs. 6.80 lakh average. Junior Rs. 3.5–6.5 lakh per AmbitionBox. Mid-level Rs. 10–17 lakh per Glassdoor. Senior Rs. 16–28 lakh. Lead Rs. 26–45 lakh nationally with Pune ±10%.

Flutter or Native Android / iOS?

Flutter for cross-platform reach. Native for deepest single-platform fluency and largest Pune Android hiring pool. Most of our students learn Flutter first if startup-targeted, native if BFSI / consumer-tech-targeted.

Do I need a Mac for Flutter?

Mac required only if you want to publish to App Store. Android development works on Linux / macOS / Windows with WSL2. We have lab access for students who want to compile to iOS without owning a Mac.

Riverpod or Bloc?

Riverpod for new projects (modern default). Bloc for large / production apps with established Bloc patterns. We teach both with honest comparison.

Will I work on real projects?

Yes — three capstone projects, with one published to both Play Store and TestFlight (App Store).

Are weekend Flutter classes available in Pune?

Yes — Saturday and Sunday, 09:00–13:00, stretched over ~6 months instead of 3.5.

What is the fee?

Course fees range Rs. 20,000 – Rs. 90,000. Play Console (~Rs. 2,500 one-time) and optional Apple Developer (~Rs. 8,500 / year) paid by the student.

What support do I get after course completion?

Six months of placement support, referrals via our alumni network (extra emphasis on Pune startup scene), resume / LinkedIn / GitHub rewrites, salary negotiation.

Is the named trainer actually teaching?

Amol Chougule personally leads every session of every batch.

Enquire or book a free demo session. Archer Infotech, Flat No. 12, Divyadarshan Housing Society, behind Kothrud Bus Stand Road, Kothrud, Pune 411038. Phone +91 9850 678451. Email info@archerinfotech.in. Web archerinfotech.in.